

《数据结构》重点笔记

第 1 部分:

1. 概念: 数据结构, 存储结构, 逻辑结构

注: 磁盘文件管理系统是树状结构。

1.1 基本概念

- (1) 数据: 指所有能够输入到计算机中并被计算机程序处理的符号的总称 (图像声音都可以通过编码归于数据的范围), **范围大**
- (2) 数据项: 数据的不可分割的最小单元
- (3) 数据元素: **是数据的基本单位**, 有若干数据项组成, 通常作为一个整体考虑
- (4) 数据对象: 性质相同的数据元素的集合, 是数据的一个子集。

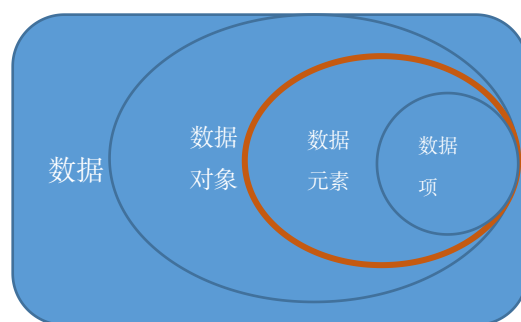
例子:

表 A

科目	分数	是否及格
数学	99	是
语文	89	是

表 B

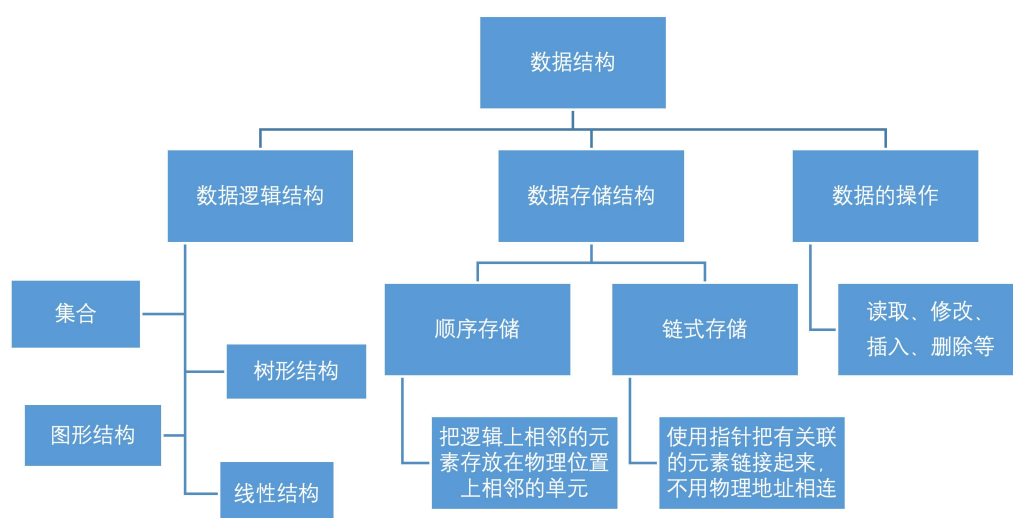
姓名	性别	身高
小花	女	100
小草	男	120



其中, A 表为成绩表, B 表为学生信息表, 这两张表就是数据; 单独的一张表就称为数据对象, 即 A 和 B 表都是一个数据对象; 每张表中的每一行就称为数据元素; 姓名, 性别, 身高, 科目, 分数就称为数据项

1.2 数据结构

定义: 相互之间存在一种或多种特定关系的数据元素的集合, 这种关系包括三方面的内容, 即**数据逻辑结构**, **数据存储结构**, **数据的操作**。

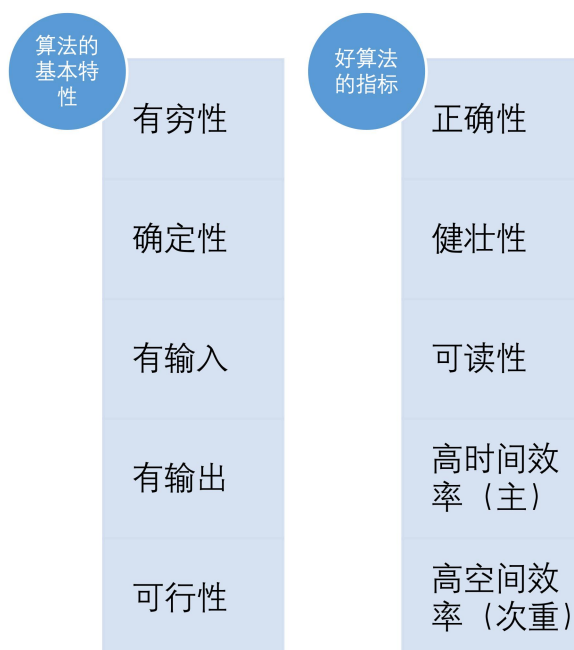


2. 数据元素是组成数据的基本单位

3. 算法，算法分析，算法特性，时间复杂度

3.1 算法：描述求解问题方法操作步骤的集合。

(不是所有的程序都是算法，要满足五个特性)



3.2 时间复杂度

3.2.1 定义：在算法分析中，一般用算法中的语句的执行次数来度量算法的时间效率，时间效率也就是时间复杂度。

3.2.2 计算方法：对于问题规模为 n 的某个函数 $f(n)$ ，算法时间复杂度记为 $T(n)$ ，存在一个正常数 c ，使 $cf(n) > T(n)$ 恒成立，则 $T(n) = O(f(n))$ ，称 $O(f(n))$ 为时间复杂度。

时间复杂度的大 O 表示法：保留最高次数项，令最高次数项的系数为 1。例如 $O(8) \rightarrow O(1)$ ， $O(2n^3 + 2n^2) \rightarrow O(n^3)$ ， $O(n * \log_2 n)$

第 2 部分

1. 线性表的概念，特点，存储结构

1.1. 1 线性表的概念：线性表是最简单，最常见，最基本的一种线性结构（数据的逻辑结构的一种），元素之间为线性关系，即除了第一个和最后一个元素之外，**所有的元素都有前驱和后继元素，同一个线性表中的数据类型相同。**

当数据元素为 0 时，可以是空表，但是没有空图的说法。

1.1.2 线性表的特点：插入删除算法的时间复杂度为 $O(n)$ ，不适合多次插入或删除

1.2.1 线性表的存储方式：

(1) **顺序存储**（又称顺序表）物理地址相连

a0	A1	A2	A3	A4	A5	A6	A7	A8	A9
----	----	----	----	----	----	----	----	----	----

(2) **链式存储**（又称链式表）

数据域	指针域
-----	-----

1.2.2 链式表的特点：可动态分配空间，插入删除算法的时间复杂度为 $O(1)$

单链表在特定节点插入和删除元素之前需要遍历链表，所以算法的时间复

复杂度为 $O(n)$ ，对于单链表的求表长，取元素，插入删除，释放掉几个操作的平均时间复杂度都是 $O(n)$ ，但是链表只需比较元素不用移动元素，所以时间效率上优于顺序表

2. 顺序存储时插入、删除（移动元素的个数）

(1) 定义顺序表的数据类型: `typedef struct{
 Int list[maxsize];
 Int size };`

(2) 初始化 `void listinitial(sequence *L){
 L->size=0; }`

(3) 插入元素: 在 i 位置前插入一个新的元素，原顺序表中 i 位置之后的元素都要移动，并修改 $L->size+1$;

```
Int insert(sequence *L, int i, int x){  
    int j;  
    if(L->size)>=maxsize {printf("线性表已满")}  
    else if (i<0||i>L->size) { printf("参数不合法")}  
    else {  
        for(j=L->size;j>i;j++) //j 指向顺序表最后一个元素 size-1 的后面  
            {L->list[j]=list[j-1];} //先逐个移动元素  
        L->list[i]=x; //再插入新的元素  
        L->size++; //使得线性表的总长度+1  
    }  
}
```

插入算法的时间效率: 平均移动元素的次数看做 $n/2$ (即移动一半元素), 时间复杂度为 $O(n)$

(4) 删除元素: 删除 i 位置上的元素，将 i 后面的元素依次往前移，再令 $L->size-1$

```
int delete(sequence *L, int i, int x){  
    int j;  
    if(L->size)<=0 {printf("线性表为空，无法删除")}  
    else if (i<0||i>L->size) { printf("参数不合法")}  
    else {  
        *x=L->list[i];  
        for(j=i+1;j<L->size-1;j++) //j 指向要删除的元素的后一位  
            {L->list[j-1]=list[j];} //再将 i 后面的元素逐个前移  
        L->size--;  
    }  
}
```

删除算法的时间效率: 任意删除一个元素平均需要移动的个数看做 $n/2$ (即移动一半元素) 时间复杂度为 $O(n)$

(5) 取元素 `*x=L->list[i];`

3. 链式存储的插入，删除的语句如何写，如何判空。

(1) 定义链式表:

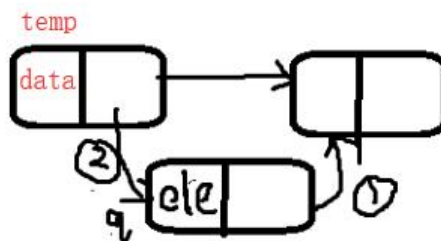
```
typedef struct Node  
{  
    int data; //这样定义就是定义了一个数组 相当于 int data[]
```

```

struct Node *next;
} Lnode,*LinkList; //Lnode 等价于 struct node 定义了一个新的数据类型
//Lnode *L 等价于 LinkList L 定义一个指向节点的指针变量

```

(2) 插入元素



```

Lnode *q=(Lnode *)malloc (sizeof(Lnode)); //分配一个新的节点空间
q->data=ele; //是该结点的数据域赋值为 ele
q->next=temp->next; //把插入的节点的指针域指向下一个节点的数据域
temp->next=q; //然后让要插入的节点的前一个节点的指针域
//指向该插入节点的数据域，两者的顺序不能乱，否则的话就丢失了插入节点的后
//一个节点的指针，找不到他了

```

(3) 删除元素



```

Lnode * del=temp->next; //单独设置一个指针指向被删除结点以防丢失
temp->next=temp->next->next; //删除 del的方法就是更改 del-1 的指针域,
//让他指向 del+1 的数据域，一步成功
free(del); //释放删除的节点 del 的空间

```

(4) 单链表判空

- 1、不带头结点的单链表 first 为空的判定条件: first=NULL;
- 2、带头结点的单链表 first 为空的判定条件: first->= NULL;

4. 顺序存储与链式存储的优缺点。

顺序表的存储优点是: 方法简单, 容易实现; 不用为表示节点间的逻辑关系而额外增加储存开销; 具有按元素序号随机访问的特点

顺序表的存储缺点是: 对于 n 较大的顺序表, 插入和删除元素需要移动的个数较大, 效率低; 需要预先分配足够大的空间, 可能会导致大量闲置空间或者出现数据溢出;

链式表的优缺点与顺序表相反。

在选择存储方式时, 若 n 较大, 多进行插入删除操作, 建议用链式存储, 若按序号访问元素时, 建议按顺序表存储 (时间效率为 O(1))

例题:

1. 在一个长度为 n 的顺序存储线性表中, 向第 i 个元素($1 \leq i \leq n+1$)之前插入一个新元素时, 需从后向前依次后移_____个元素。

- A、 $n-i$ B、 $n-i+1$ C、 $n-i-1$ D、 i

10. 从一个长度为 n 的顺序表中删除第 i 个元素($1 \leq i \leq n$), 需向前移动 () 个元素。

- A. $n-i$ B. $n-i+1$
C. $n-i-1$ D. i

7. 在一个长度为 n 的线性表中插入第 i 个元素的操作中, i 的取值范围是

- A. $1 \leq i \leq n$ B. $0 \leq i \leq n$ C. $1 \leq i \leq n+1$ D. $1 \leq i \leq n-1$

2. 在一个长度为 n 的顺序表中删除第 i 个元素($0 \leq i \leq n$)时, 需向前移动 _____ 个元素。

- A. $n-i$ B. $n-i+1$ C. $n-i-1$ D. i

第 3 部分

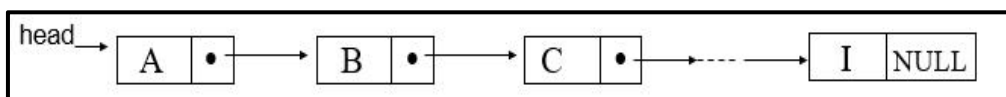
1. 字符串的基本操作, 什么叫模式匹配、

1.1 串的定义: 字符串 (也就是串) 是 0 个或多个字符序列组成的有限序列。

- 串就是数据元素为单个字符的特殊线性表。
- “qhjdkcbjsb” (隐含结束符 0) 就是一个字符串, 引号起界定作用, “” 表示空串, 串长为 0
- “ ” 空格串不是空串, 串长为空格数

1.2 串的存储类型: **顺序存储**: 定长顺序存储结构、堆分配存储结构 (动态分配连续内存)

链式存储

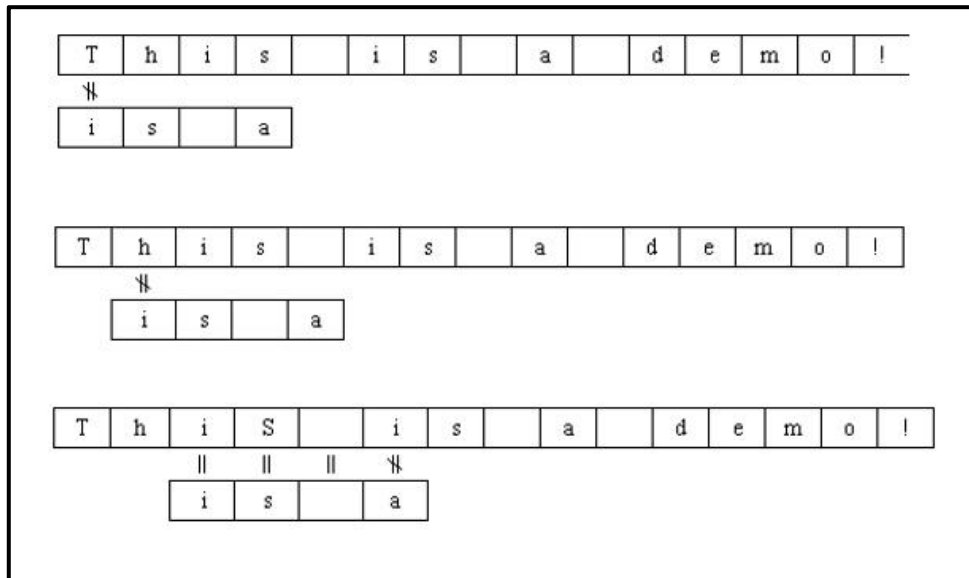


1.3 串的模式匹配: 就是指定一个主串 S 和子串 T , 求 T 在 S 中第一次出现的位置。

(1) 朴素模式匹配 (BP 算法)

核心思想: 主串 S 和子串 T , $S[1]$ 和 $T[1]$ 比较, $S[2]$ 与 $T[2]$ 比较, 直到 $S[n]$ 与 $T[n]$ 都是相等的为止, 若 $S[i]$ 与 $T[i]$ 不相等, 则子串向右移动一个 (一个字符) 继续比较

时间复杂度: 主串 S 长度为 n , 子串 T 长度为 m , 最多进行 $m(n-n+1)$ 次, 最坏的时间复杂度为 $O(mn)$ 效率不高



(2) KMP 算法

核心思想：尽量利用已经得到的“部分匹配”的结果信息，不要让 i 回溯，加快子串右滑的速度，**问题由子串决定而不是主串决定的**

next 数组：是一个智能数组，指导子串下一步改用几号元素去匹配，next 数组的 next[1]和 next[2]永远为 0 和 1，其余 next[n]看 n 位的前后缀的相同数目

实例：(1) 现有子串 i=“ABABAAABA”,求其 next 数组

i	9	a	b	a	b	a	a	a	b	a
j	0	1	2	3	4	5	6	7	8	9
next		0	1	1	2	3	4	2	2	3

Next 数组中对应 i 数组的值	前缀	后缀	Next[]的值
Next[j1]	0	0	0
Next[j2]	A	A	1
Next[j3]	A	B	1
Next[j4]	A	A	2
Next[j5]	AB	AB	3
Next[j6]	ABA	ABA	4
Next[j7]	A	A	2
Next[j8]	A	A	2
Next[j9]	AB	AB	3

(2) 实例，不详细解释

主串是 S=“bbsbbcdffb” 子串是 “bbsbbs”,求 next 数组

T	6	b	b	s	b	b	s
J	0	1	2	3	4	5	6
next		0	1	2	1	2	3

主串是 S=“ssssvsd”,子串是 T=“ssssb” 求 next 数组

S	5	s	s	s	s	B
I	0	1	2	3	4	5
Next		0	1	2	3	4

Next 数组中对应 i 数组的值	前缀	后缀	Next[]的值
Next[j1]	0	0	0
Next[j2]	s	s	1
Next[j3]	s	s	2
Next[j4]	ss	ss	3
Next[j5]	sss	sss	4

KMP 算法的时间复杂度: $O(m+n)$

第 4 部分

1. 栈的基本概念与特点

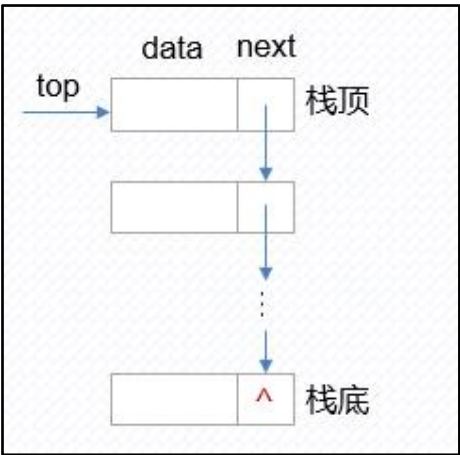
栈和队列也是线性表,只是操作受限制的线性表,他们线性表的区别在于线性表可以在任意位置插入或删除元素,而栈只能在栈顶操作,队列只能在队尾操作。

1.1 栈的特点: 后进先出 (LIFO) 可看做装电池,只能对栈顶进行操作

1.2 存储方式: 顺序存储和链式存储



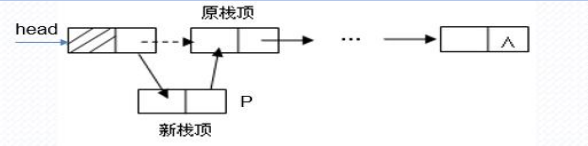
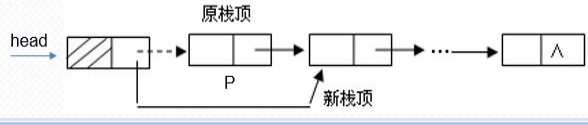
有 n 个元素的栈的顺序存储



栈的链式存储

2. 栈的两种存储结构的基本操作, 如何入栈, 出栈, 判空?

操作	栈	
存储方式类型	顺序栈	链式栈

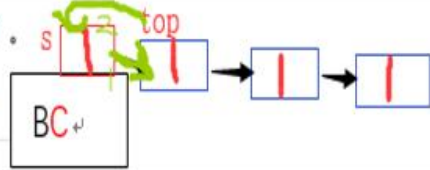
定义数据类型	<pre> Typedef struct{ Int stack[maxsize]; Int top; }sequencestack; </pre>	<pre> Typedef struct snode{ Int data; Int snode *next; }singlelinkednode; </pre>
初始化	S->top=0;	
取栈顶	*d=S.stack[S.top-1];	<pre> Singlelinkednode *p=head->next; *d=p->data; </pre>
入栈	<pre> S->stack[S->top]=x; S->top++; </pre> 移动 top 指针	<pre> Singlelinkednode *p; p->data=x; p->next=head->next; //先执行 head->next=p; //后执行 </pre> 
出栈	<pre> S->top--; </pre> 直接移动 top 指针, 不用删除原栈顶	<pre> head->next=head->next->next; free(p); //需要释放出栈的元素 </pre> 
判空	<pre> if(S->top==0) printf("stack is empty"); </pre>	<pre> If(head->next==null) Printf("stack is empty"); </pre>

例题:

*7. 向一个栈顶结点为Top的链中插入一个s所指结点时, 其操作步骤为()。

(A) Top->next=s; (B) s->next=Top->next; Top->next=s;
 (C) s->next=Top; Top=s; (D) s->next=Top; Top=Top->next;

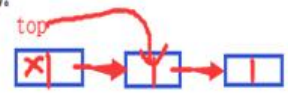
BC



*8. 从链栈Top中删除一个结点, 并将被删结点的值保存到x中, 其操作步骤为()。

(A) x=Top->data; Top=Top->next;
 (B) Top=Top->next; x=Top->data;
 (C) x=Top->next->data; Top->next=Top->next->next;
 (D) x=Top->data; Top->next=Top->next->next;

CA



在链栈中也只能删除栈顶元素top, 不要和线性表的删除元素弄混了

3. 栈的应用举例 (进制转换, 递归, 迷宫, 表达式求值等)

栈的应用	内容
进制转换	算法原理: 辗转相除法 算法公式: $N=(N \text{ div } 2) * 2 + N \text{ mod } 2$

	<table><tr><th>N</th><th>N div 2</th><th>N mod 2</th></tr><tr><td>599</td><td>299</td><td>1</td></tr><tr><td>299</td><td>149</td><td>1</td></tr><tr><td>149</td><td>74</td><td>1</td></tr><tr><td>74</td><td>37</td><td>0</td></tr><tr><td>37</td><td>18</td><td>1</td></tr><tr><td>18</td><td>9</td><td>0</td></tr><tr><td>9</td><td>4</td><td>1</td></tr><tr><td>4</td><td>2</td><td>0</td></tr><tr><td>2</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	N	N div 2	N mod 2	599	299	1	299	149	1	149	74	1	74	37	0	37	18	1	18	9	0	9	4	1	4	2	0	2	1	0	1	0	1
N	N div 2	N mod 2																																
599	299	1																																
299	149	1																																
149	74	1																																
74	37	0																																
37	18	1																																
18	9	0																																
9	4	1																																
4	2	0																																
2	1	0																																
1	0	1																																
递归																																		
迷宫																																		
表达式求值	例 1: $A+(B-C/D)*E$ （也称为中缀表达式） 前缀表达式为: $+A*-B/CDE$ 即运算符在数值前面 后缀表达式为: $ABCD/-E*+$ 运算符在数值后面 例 2: $[(a+b)+c*(d+e)+f]*(g+h)$ 前缀表达式为: $*++ab*+cdeg+gh$ 后缀表达式为: $ab+cde+*+f+gh+*$																																	

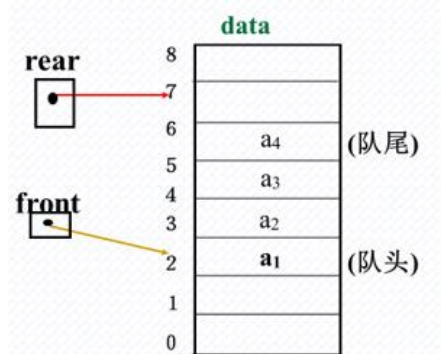
例题:

- 7、将递归算法转换成对应的非递归算法时, 通常需要使用(B)来保存中间结果。
- A. 队列 **B. 栈** C. 链表 D. 树
- 3、设计一个判别表达式中括号是否配对的算法, 采用(D)数据结构最佳。
- A. 顺序表 B. 链表 C. 队列 **D. 栈**

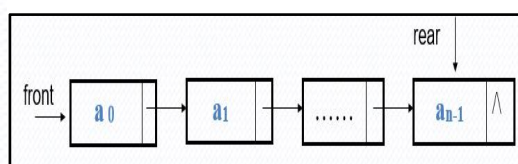
4. 队列的基本概念与特点

4.1 队列的特点: 先进先出 (FIFO) 可看做排队买冰淇淋, 只能对队头和队尾进行操作

4.2 存储方式: 顺序存储和链式存储 (注意 front 和 rear 的位置)



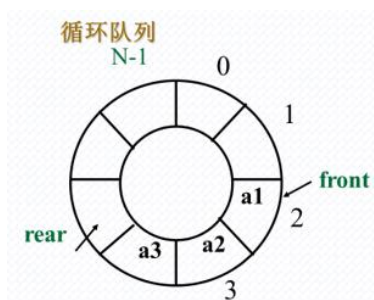
队列的顺序存储



队列的链式存储

4.3 假溢出问题

队列中因为多次出队入队而导致的存储空间不足的问题——解决办法: 循环队列



$$\text{front}+1 == [\text{front}+1] \% \text{maxsize};$$

5. 队列的两种存储结构的基本操作，队空，队满，环形队列，假溢出？

队列的操作	循环队列	链式队列
定义数据类型	<pre> Typedef struct{ Int queue[maxsize]; Int rear,front,count; }sequencequeue; </pre>	定义节点类型 <pre> Typedef struct qnode{ Int data; Struct qnode *next; }linkedqueuenode; </pre> 定义队列的结构体 <pre> Typedef struct{ Linkedqueuenode *front; Linkedqueuenode *front; }Lqueue; </pre>
判空与判满	方法一：使用 count 记录元素个数 队列已满 count>0&&rear==front; 或 count==maxsize; 队列已空 count==0; 方法二：加设标志位出队时置0入队时置1 队列已满：tag==1 && rear==front 队列已空：tag==0 && rear==front 方法三：用掉一个存储单元（默认） 队列已满： <u>front=(rear+1)%MaxQueueSize</u> 队列已空：rear==front	If(Q.front==null) Printf(“”queue is empty); 链式队列不会满
入队(在队尾入队)	<pre> Sequencequeue Q; Q->queue[Q->rear]=x; Q->rear=(Q->rear+1)% maxsize; </pre>	<pre> P=(linkedqueuenode *)malloc(sizeof(linkedqueuenode)) Q->rear=p; p->next=null; </pre>
出队（在队头出队）	<pre> *d=Q->queue[Q->front]; Q->front=[Q->front+1]%maxsize; </pre>	<pre> P=Q->front; Q->front=Q->front->next; Free(p); </pre>
取队头元素	<pre> *d=Q->queue[Q->front]; </pre>	<pre> *d=Q.front->data; </pre>

例题：

★3. 循环队列的入队操作应为()。

(A) sq.rear=sq.rear+1; sq.data[sq.rear]=x;

(B) sq.data[sq.rear]=x; sq.rear=sq.rear+1;

(C) sq.rear=(sq.rear+1)% maxsize; sq.data[sq.rear]=x;

(D) sq.data[sq.rear]=x; sq.rear=(sq.rear+1)% maxsize;

D

6. 队列的应用举例（CPU 对资源的管理，图的广度周游等，树的层次周游）

7. 写算法：从把一个队列中的元素存储到栈。将一个栈中的元素放于队列中的算法（写算法要求会写程序，核心语句一定要会）（可以直接调用 push pop 操作）

1. 把队列中的元素存到栈中

- (1) 新建队列与栈并赋值

```
Q=createEmptyQueue_seq();//队列  
S= createEmptyStack_seq();//栈
```

- (2) 取队头: `n=frontQueue_seq(Q);`

- (3) 将队头入栈顶: `push_seq( S, n );`

- (4) 出队: `popQueue_seq(Q);`

2. 把栈中的元素存到队列中

- (1) 取出栈顶: `n=top_seq( S);`

- (2) 将栈顶入队尾: `pushQueue_seq(Q,n);`

- (3) 出栈: `popStack_seq(S);`

第 5 部分

1. 二叉树的概念，性质 1-6

1.1 树：即树形结构，是数据逻辑结构的一种。

- (1) 特点：每个结点最多一个前驱结点，多个后继结点
- (2) 度的概念：结点的度就是结点的孩子数，树的度就是所有结点的度的最大值。
- (3) 深度的概念：根的层次为 0，依次往下数。
- (4) 实例：磁盘文件的目录结构。
- (5) 有序树与无序树：即对子树是否加以区别。

1.2.1 二叉树：结点有限，区分左右子树的有序树。二叉树不是树的特殊情况，树与二叉树的区别在于二叉树是有序的。

1.2.2 二叉树的分类

满二叉树



- ☐ 所有节点的度为0或2
- ☐ 所有叶子节点在同一层

完全二叉树



- ☐ 只有树的最后两层的结点的度可以小于2
- ☐ 最后一层的叶子节点从左到右填充

1.2.3 二叉树的性质

一颗非空二叉树的第i层上至多有 2^i 个节点

高度为k的二叉树最多有 $[2^{k+1}]-1$ 个节点(就是满二叉树的状态)

若二叉树的叶子节点数为 n_0 ，度为2的节点数为 n_2 ，则 $n_0=n_2+1$

• 叶子节点度为0，则度为0,1,2 的节点数为 n_0, n_1, n_2 ， $n=n_0+n_1+n_2$ ，总的边数 $B=n-1=0*n_0+1*n_1+2*n_2$ ，联立解得 $n_0=n_2+1$

具有n个节点的完全二叉树的高度k为 $\lceil \log_2 n \rceil$

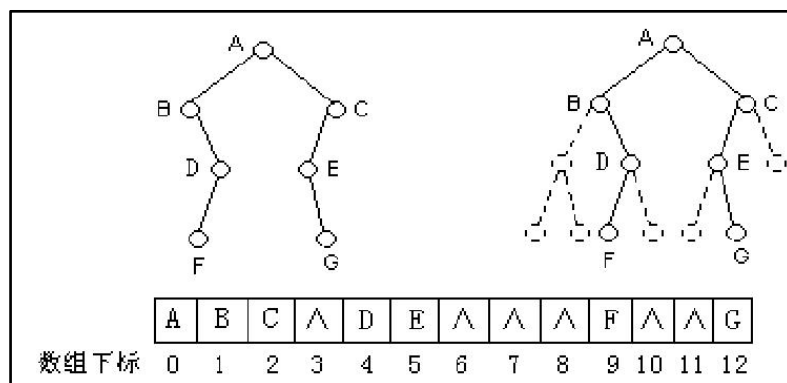
有n个结点的完全二叉树若节点按层次编号(0开始)，则对于序号为i的节点有

- 若 $2i+1 \leq n-1$ ，则结点i左孩子结点的序号为 $2i+1$ ，否则没有左孩子
- 若 $2i+2 \leq n-1$ ，则结点i右孩子结点的序号为 $2i+2$ ，否则没有右孩子

在满二叉树中，叶子节点的个数比分支节点的个数多1

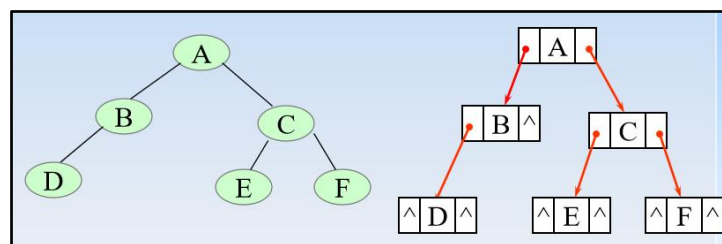
1.2.4 二叉树的存储方式

- (1) **顺序存储**: 采用一组连续的存储单元存放二叉树的节点，没有节点的地方需要补上。



- (2) **链式存储**: 用一条链表储存二叉树，一个节点有数据域和两个左右孩子的指针组成，若没有左/右孩子用^表示

lchild	data	rchild
--------	------	--------



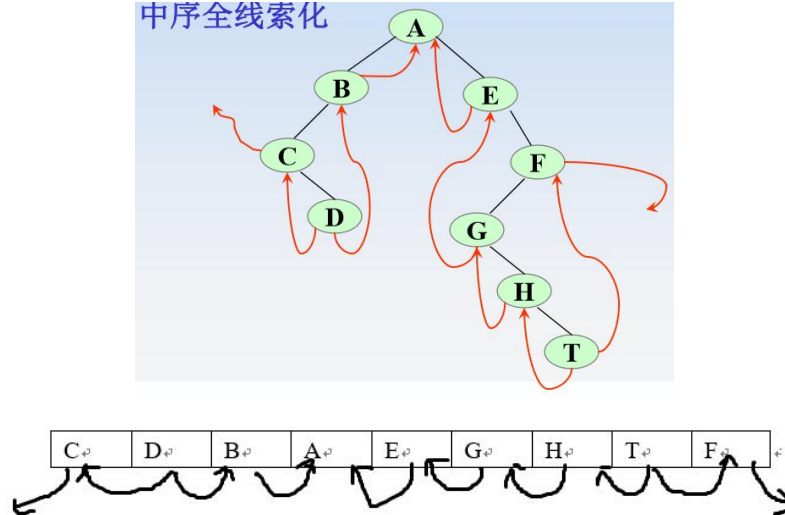
1.3 线索二叉树

- 1、定义: 增加节点标志域，利用这些指针把节点的前驱和后继的节点的信息连接起来，建立线索二叉树，主要是为了保留在各种遍历时得到的节点前驱和后继节点信息（即保留在各种遍历时产生的线性关系）。

Lefttag	lchild	data	rchild	righttag
---------	--------	------	--------	----------

- 2、实例: 中序周游的结果是: CDBAEGHTF,有边相连就无需虚线相连，无边相连则用虚线连起来

中序全线索化



2. 二叉树的周游，中，先—>后，画树

	先序周游	中序周游	后序周游
顺序	根左右	左根右	左右根
实例	*+ab-cd	A+b*c-d	Ab+cd-*
递归运算	<pre>void postOrder(BNode p) { if (p == NULL) return; visit(p); postOrder(leftChild_btree(p)); postOrder(rightChild_btree(p)); }</pre>	<pre>void postOrder(BNode p) { if (p == NULL) return; postOrder(leftChild_btree(p)); visit(p); postOrder(rightChild_btree(p)); }</pre>	<pre>void postOrder(BNode p) {if (p == NULL) return; postOrder(leftChild_btree(p)); postOrder(rightChild_btree(p)); visit(p);}</pre>

3. 二叉树的存储结构，写出链式存储结构

4. 写算法：二叉树的中序，先序，后序的递归算法，求叶子结点算法 求叶子结点的算法代码：

```
Void  contleaf(BiTree T,int  *count){
If(T){If(!T->lchild)&&(!T->rchild))
    *Count)++; //计算根节点
    Countleaf(T->lchild ,count); //递归调用求左子树的节点
    Countleaf(T->rchild ,count); //递归调用求右子树的节点
}}
```

5. 哈夫曼算法及哈夫曼编码，WPL 的计算

5.1 哈夫曼树相关概念

- (1) 路径长度：从 A->B 所经过的分支序列称路径，经过的分支个数称为路径长度
- (2) 带权路径长度：若二叉树中的叶节点都带有权值，从根节点到叶节点的路径长度与相应的叶节点权值乘积之和称为该树的**带权路径长度 (WPL)**

$$WPL = \sum_{i=1}^n w_i \cdot l_i$$

- (3) 哈夫曼树：一组带权的叶节点可以构造多个二叉树，其中 WPL 最小的树称为**哈夫曼树或最优二叉树**。

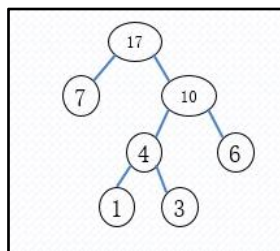
- (4) 假设哈夫曼树叶子节点的个数为 m，则哈夫曼树的分支节点为 m-1，总共节

点个数为 $2m-1$

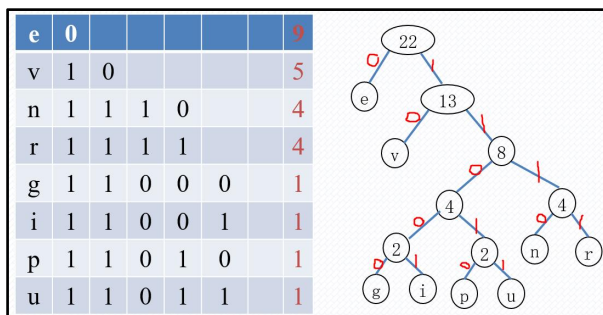
5.2 哈夫曼树的构造：原则是将权值大的结点靠近根。

根节点的值等于所有叶子节点的权值之和

例子：用 7, 10, 1,3,4,6,17 构造一棵哈夫曼树



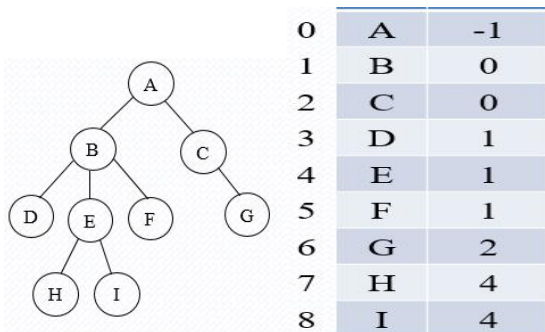
5.3 哈夫曼树编码：在哈夫曼树中把每个节点引向左孩子上的边标为 0，引向右孩子的边标为 1，再从上至下写出节点的编码



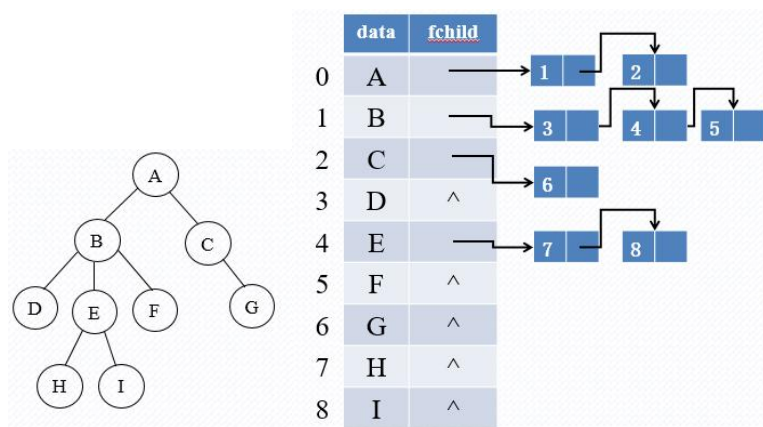
6. 二叉树与树的转换

6.1 树的表示方法

6.1.1 父指针表示法

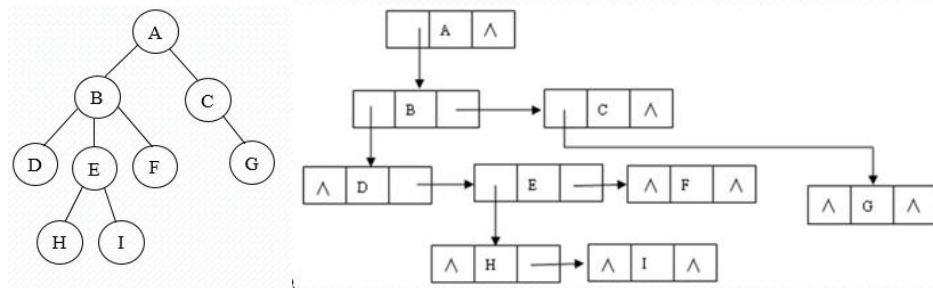


6.1.2 孩子链表表示法



6.1.3 孩子兄弟表示法

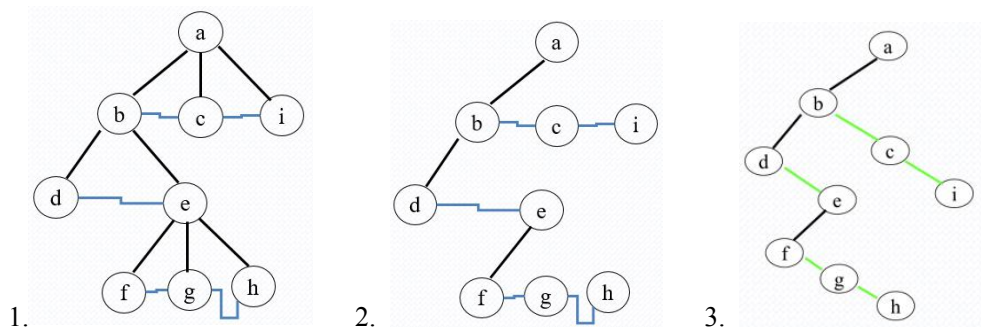
增加两个分别指向该结点的第一个孩子结点和下一个兄弟结点的指针，即**左分支存放孩子节点，有分支存放兄弟节点**



6.2 树与二叉树的转换

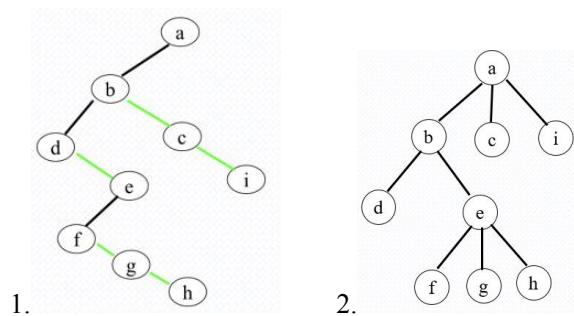
6.2.1 树转为二叉树

- 1、树中同一结点的兄弟相连；
- 2、保留结点的最左孩子连线，删除其它孩子连线；
- 3、调整位置；



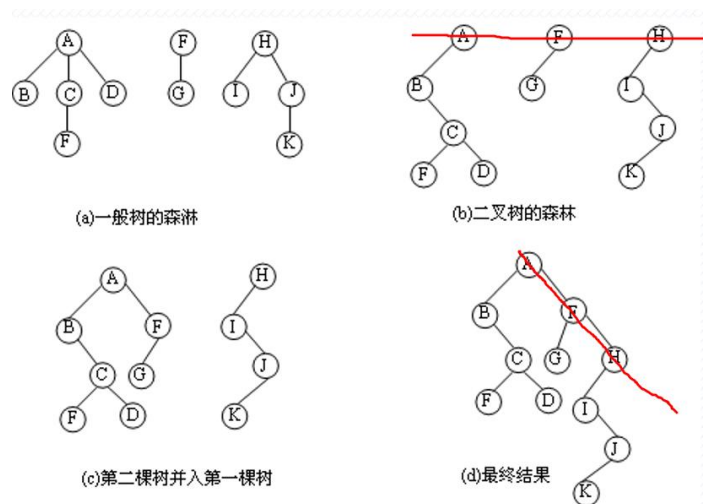
6.2.2 二叉树转为树

即把所有右孩子节点变为兄弟节点



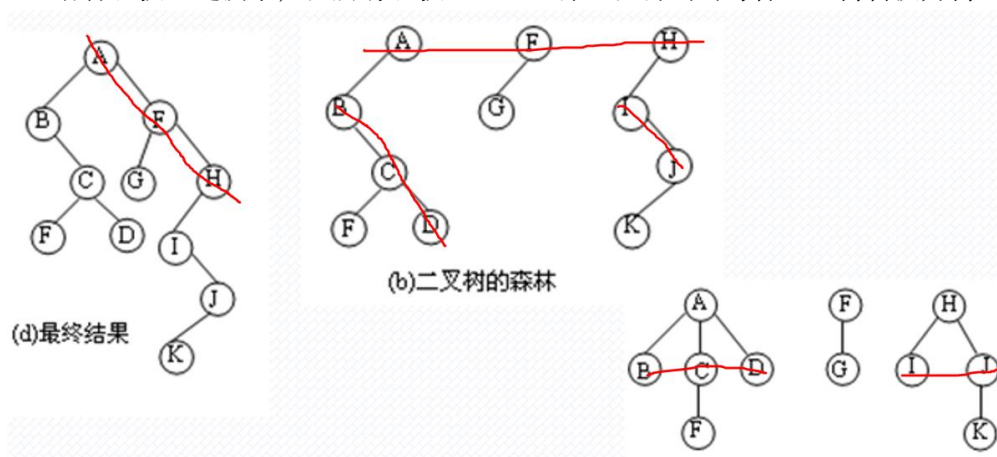
6.2.3 森林转为树

森林中各树先各自转为二叉树;依次连到前一个二叉树的右子树上



6.2.4 树转为森林

沿着右孩子链搜索，把所有右孩子连线去掉；把得到的每棵二叉树转换为树



第 6 部分

1. 图的概念、图的存储结构，由图写矩阵或由矩阵画图，矩阵的特点，时空复杂度。

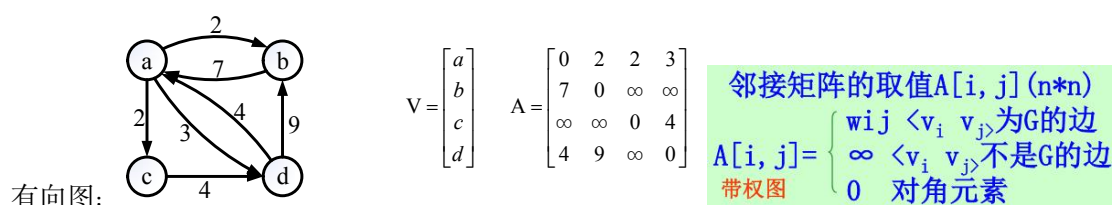
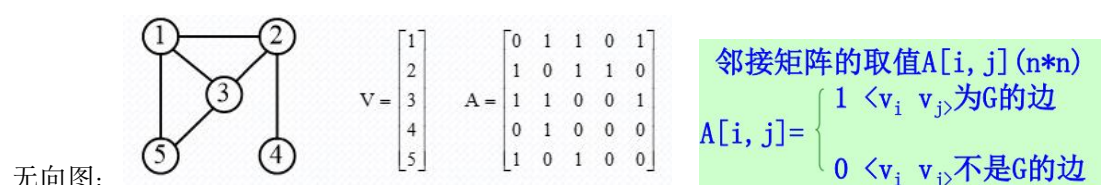
1.1 图的概念：由顶点集合及顶点间的关系集合组成的一种数据结构。图是一种非线性结构，图形结构是数据的逻辑结构的一种，节点使一对多的关系，不具有明显的分层关系。

图的基本术语	解释及注意事项
无向图	若 n 个顶点的无向图有 $n(n-1)/2$ 条边，称为无向完全图
有向图	若 n 个顶点的有向图有 $n(n-1)$ 条边，称为有向完全图
无向完全图	在无向图中任意两个顶点之间都有连线
有向完全图	在有向图中任意两个顶点之间都有连线
度	顶点的度=出度+入度
出度	顶点指出去的边
入度	指向顶点的边
权值	权值可以表示从前一个工程到后一个工程所需的时间或者代价
路径长度	路径 (path) 上边或者弧的数目
简单路径	若一条路径上顶点不重复出现就称为简单路径

简单回路	除第一个和最后一个顶点相同外，其余点不重复的回路称简单回路
回路	第一个顶点和最后一个顶点相同的回路或环
连通图	无向图中任意两顶点之间是有连通的，没有孤立点称为连通图
连通分量	无向图的最大连通子图称为连通分量
强连通图	有向图中任意两顶点之间是连通的称为连通图
强连通分量	有向图的 最大 连通子图称为连通分量
极小连通子图	是指在包含所有顶点并且保证连通的前提下包含原图中 最少 的边。
生成树	包含图的全部顶点的一个极小连通子图

1.2 邻接矩阵——顺序存储

- (1) 邻接矩阵：用两个数组表示图，一个是存储图中顶点信息的一维数组，一个是存储边信息的二维数组。

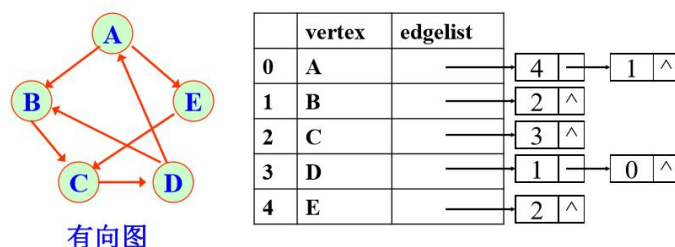


- (2) 邻接矩阵的特点:

- 无向图的邻接矩阵是一个对称矩阵。存放时只需要存放上三角矩阵的元素即可
- 查找图中任一顶点的度方便，易于判定任意两个顶点之间是否有边相连。对于无向图而言，顶点 v_i 的度就是邻接矩阵中第 i 行或第 i 列中非 0 或非 ∞ 的元素个数。对于有向图而言，顶点 v_i 的入度是邻接矩阵中第 i 列中非 0 或非 ∞ 的元素个数，顶点 v_i 的出度是邻接矩阵中第 i 行中非 0 或非 ∞ 的元素个数。
- 查找图中任一条边或弧的权值方便。

1.2 邻接表——顺序存储和链式存储的结合

- (1) 用一个一维顶点数组存储顶点信息，每个顶点元素有 data 域和指针域，指针域存放该顶点的邻接表的第一个节点的地址。



- (3) 空间复杂度:

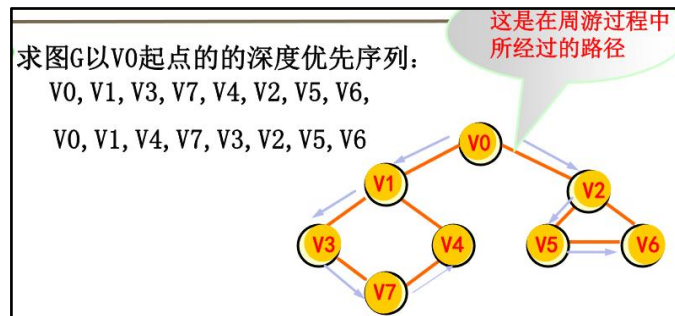
- a、当用二维数组表示邻接矩阵作图的存储结构时，查找每个顶点的邻接点所需时间为 $O(n^2)$,其中 n 为顶点数.
- b、当以邻接表作图的存储结构时，找邻接点所需时间为 $O(e)$,其中 e 为无向图中边的数或有向图中弧的数.
- c、当以邻接表作为存储结构时，深度优先遍历图的时间复杂度为 $O(n + e)$.

2. 图的周游/遍历 (注意其存储结构)

遍历: 从图的某个顶点出发, 按照一定的顺序访问图中的每个顶点, 且每个顶点有且仅有一个被访问。

2.1 深度优先遍历: 类似与树的先序遍历, 遍历的结果不唯一。通常采用栈实现

解释: 从图中某个顶点 v 出发, 访问此顶点, 然后依次从 v 的未被访问的邻接点出发深度优先遍历图, 直至图中所有和 v 有路径相通的顶点都被遍历过。若此时图中尚有未被访问的顶点, 则另选图中一个未被访问的顶点作为起始点, 重复上述过程, 直到图中所有顶点都被访问到为止。



*4. 用邻接表表示图进行深度优先遍历时, 通常是采用 () 来实现算法的。
 (A) 栈 (B) 队列 (C) 树 (D) 图

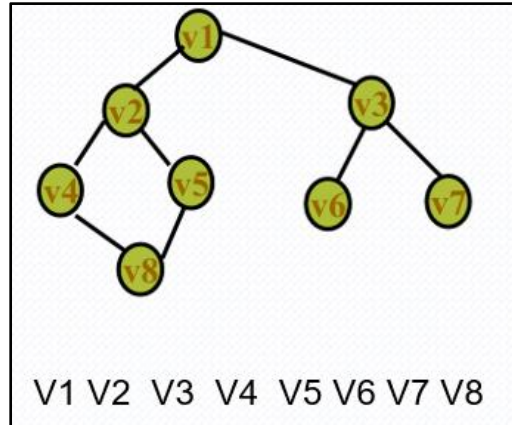
CA

*6. 图的深度优先遍历类似于二叉树的 () 。
 (A) 先序遍历 (B) 中序遍历 (C) 后序遍历 (D) 层次遍历

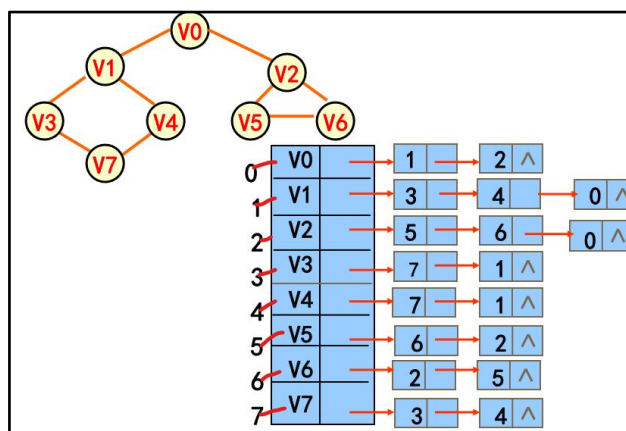
DA

2.2 广度优先遍历: 类似于树的层次遍历, 采用队列实现

解释: 在访问了起始点 v 之后, 依次访问 v 的邻接点; 然后再依次 (顺序) 访问这些点 (下一层) 中未被访问过的邻接点; 直到所有顶点都被访问过为止。



2.3 两种遍历算法的区别



对于上图的遍历中，深度优先遍历得到的结果是：v0,v1,v3,v7,v4,v2,v5,v6

广度优先遍历得到的结果是：v0,v1,v2,v3,v4,v5,v6,v7

3. 最小生成树算法：PRIM 与 KRUSKAL 算法

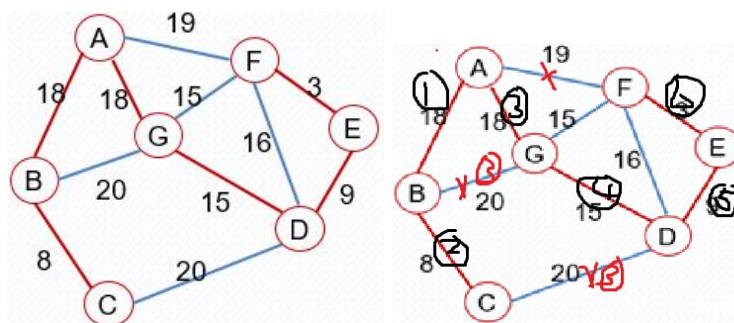
最小生成树：连通图的生成树中权值总和最小的生成树，称为最小代价生成树(MST)

特点：必须包含所有(n)个顶点；最小生成树有且仅有 n-1 条边；不存在回路。

构造方法：prim 算法和 kruskal 算法。

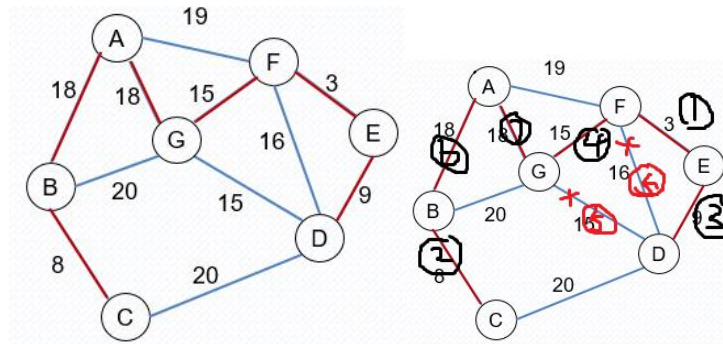
(1) prim 算法：

假设图 $G=(V,E)$ 中已落在生成树上的顶点集为 U ，则尚未落在生成树上的顶点集为 $V-U$ ，从 $(V-U)$ 顶点集中选取顶点 w 加 U 集合，顶点 w 需满足下列条件：它和生成树上的顶点之间的边上的权值是在联接这两类顶点的所有边中权值最小。



(2) kruskal 算法：

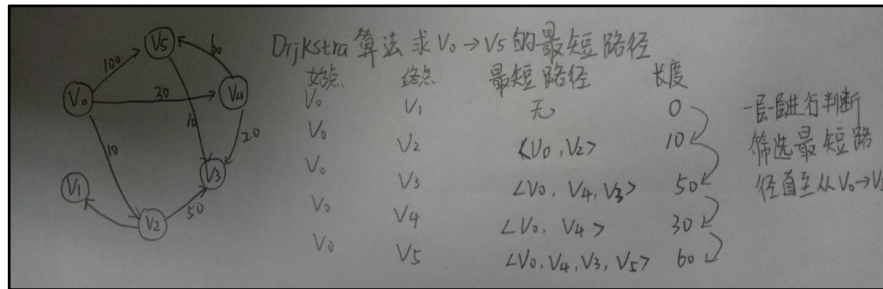
为使生成树上总的权值之和达到最小，则应使每一条边上的权值尽可能地小，自然应从权值最小的边选起，直至选出 $n-1$ 条互不构成回路的权值最小边为止



4. 最短路径算法: Dijkstra 算法

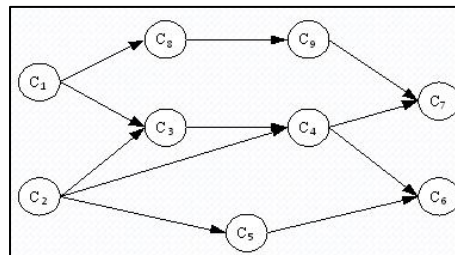
最短路径的概念: 即弧的权值之和取最小值的那条路径

算法思想: 按各条最短路径长度递增的次序, 产生最短路径的算法。

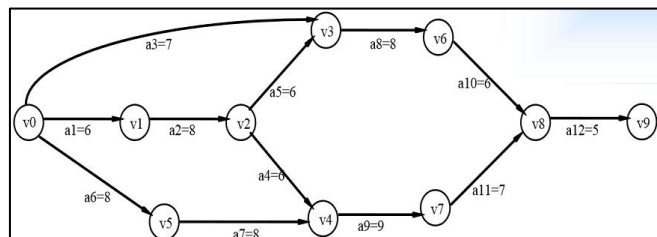


5. 拓扑排序、关键路径 AOE

5.1 AOV 网: 当有一些活动必须在其他活动完成之后才能开始, 用顶点表示活动, 弧表示活动间的优先关系, 这样的有向图称为 AOV 网



5.2 AOE 网: 用顶点表示事件, 弧表示活动, 弧上的权值表示活动所需时间构造的有向无环图称为 AOE 网。



5.3 拓扑排序算法的步骤:

- 在有向图中选择一个入度为 0 的顶点, 输出该顶点
- 从图中删除所有以它为尾的弧

c、重复执行 a,b, 直到找不到入度为 0 的顶点, 拓扑排序完成。

注: 若最终图中仍有顶点存在, 不存在入度为 0 的点, 则证明有环
若拓扑排序成功则证明无环路。

5.4 **关键路径**: AOE 网中存在唯一的、入度为零的顶点, 叫做源点, 存在唯一的、出度为零的顶点, 叫做汇点。从源点到汇点的**最长路径的长度**即为完成整个工程任务所需的时间, 该路径叫关键路径。

第 7 部分

1. 字典的相关概念,ASL

(1) 什么是字典:

字典是由一些具有相同可辨认特性的数据元素 (或记录) 构成的集合。

(2) 关键字

是数据元素中某个数据项的值, 用它可以标识 (识别) 一个数据元素。

(3) 检索: 根据给定的某个值, 在查找表中确定一个关键字等于给定值的数据元素。

1. 基于线性表的检索: 顺序检索、**二分检索**

2. 根据关键码值直接访问: **散列检索**, 基于数组下标的直接检索

3. 树索引方法: **二叉搜索树**、字符树、B 树

(4) 平均查找长度 ASL——查找过程中对关键字需要执行的平均比较次数

$$ASL = \sum_{i=1}^n P_i C_i = 1/n \sum_{i=1}^n C_i \quad \text{其中:}$$

n 是数据元素的个数

Pi 是查找第 i 个数据元素概率 (通常取等概率, 即 Pi=1/n)

Ci 是找到第 i 个记录时所经历的比较次数。

(5) 查找方法

1. 静态查找

1.1 顺序表的查找

思想: 从顺序表的一端开始扫描, 将给定的元素与每个数值比较

成功查找长度 ASL: **ASL=(n+1)/2**, 若失败则 ASL=n+1

优点: 算法简单, 且对顺序结构或链表结构均适用。

缺点: ASL 太大, 时间效率太低。

存储: 顺序表查找方法既适用于线性表的顺序存储结构, 也适用于线性表的链式存储结构 (单链表)

1. 若查找每个元素的概率相等, 则在长度为 n 的顺序表上查找任一元素的平均查找长度为(**D**)。

A. n B. n+1 C. (n-1)/2 D. (n+1)/2

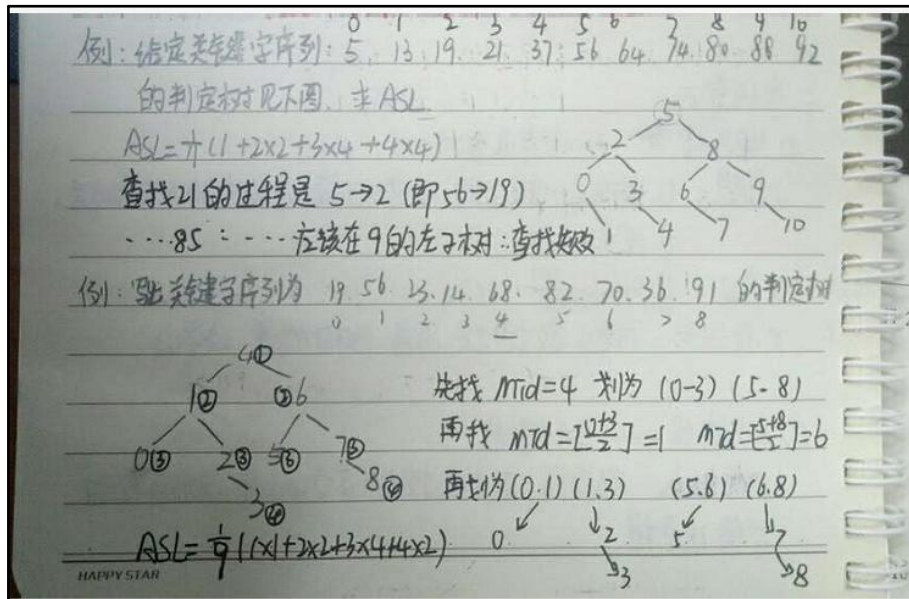
4. 从一个具有 n 个结点的单链表中查找其值等于 x 的结点时, 在查找成功的情况下, 需平均比较 **C** 个元素结点。

A. n/2 B. n **C. (n+1)/2** D. (n-1)/2

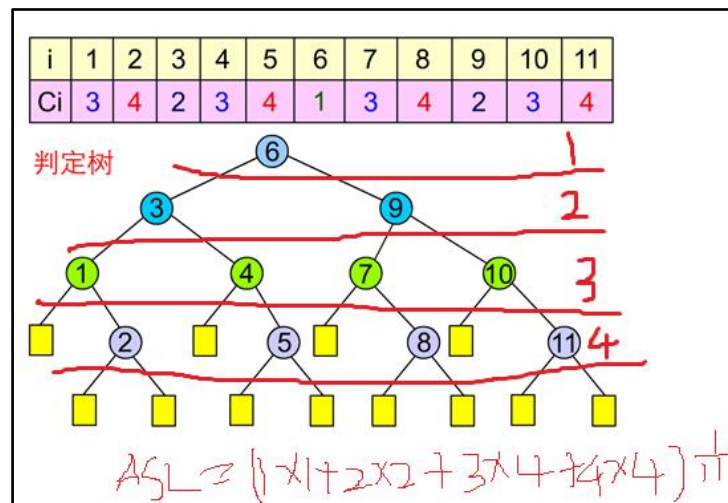
1.2 二分查找(折半查找)

(1) 基本思想: 先给数据排序 (例如按升序排好), 形成有序表, 然后再将 key 与正中元素相比, 若 key 小, 则缩小至前半部内查找; 若 key 大, 则缩小至后半部内查找。在每个区域内再取其中值比较, 每次缩小 1/2 的范围, 直到查找成功或失败为止。

- (2) 对线性表的要求: 要求线性表按关键字排序, 排序本身是一种费时的运算, 所以二分法的时间复杂度最好也是 $O(n \log 2 n)$, 增加了算法的时间复杂度
- (3) 算法 $ASL = \log 2 n$
- (4) 二分查找只适用于顺序存储结构的有序表, 链表无法实现二分查找。



- (5) 二分查找的比较次数(C_i)需要借用判定树来计算



判定树的画法: 按照比较的次数生成判定树, 比较 1 次的是根结点, 比较 2 次的在第二层, 比较 3 次的在第三层, 一次类推, 也可以说是每次的 mid 即形成判定树的结点, 左子树上的结点是有顺序前半部分的所有结点, 右子树是后半部分的结点。

判定树求平均成功平均查找长度和不成功平均查找长度的方法:

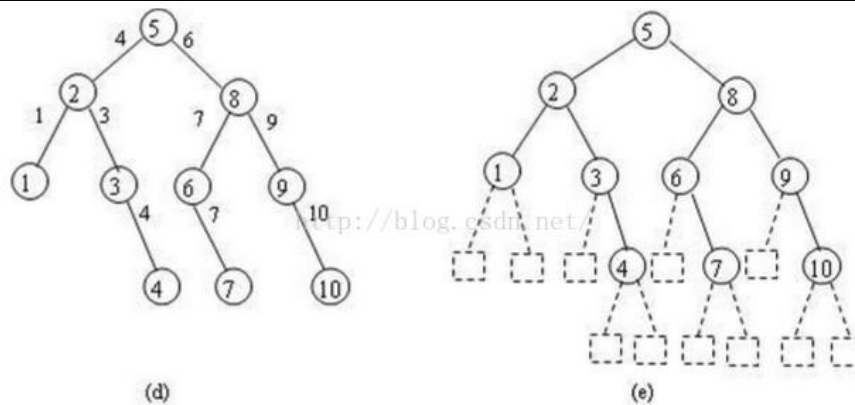


图 7-2 折半查找判定树的生成过程

对于折半查找判定树，需要补充以下两点：

- (1) 折半查找判定树是一棵二叉排序树，即每个结点的值均大于其左子树上所有结点的值，小于其右子树上所有结点的值；
- (2) 折半查找判定树中的结点都是查找成功的情况，将每个结点的空指针指向一个实际上并不存在的结点——称为外结点，所有外结点即是查找不成功的情况，如图7-2(e)所示。如果有序表的长度为 n ，则外结点一定有 $n+1$ 个。在折半查找判定树中，某结点所在的层数即是查找该结点的比较次数，整个判定树代表的有序表的平均查找长度即为查找每个结点的比较次数之和除以有序表的长度。例如，长度为10的有序表的平均查找长度为：

$$ASL = (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 3) / 10 = 29/10$$

在折半查找判定树中，查找不成功时的比较次数即是查找相应外结点时与内结点的比较次数。整个判定树代表的有序表在查找失败时的平均查找长度即为查找每个外结点的比较次数之和除以外结点的个数。例如，长度为10的有序表在查找失败时的平均查找长度为：

$$ASL = (3 \times 5 + 4 \times 6) / 11 = 39/11$$

例题：

2. 对于长度为 9 的顺序存储的有序表，若采用折半查找，在等概率情况下的平均查找长度为 (C) 的 9 分之一。

A. 20 B. 18 C. 25 D. 22

2. 二分查找对线性表的要求，会写二分检索算法。

代码如下：

```
int BinarySearch(SequenceList R, ElemType x)
{
    int low=0, high=R.size-1, mid;
    while(low<=high) {
        mid=(low+high)/2;
        if(R.list[mid].key==x.key)    //判断 mid 是否=key
            return mid;                /*查找成功返回元素位置*/
        if(R.list [mid].key>x.key)    //若 key<mid 则在 mid 左边的元素进行递归查找
            high=mid-1;                //注意此时在设定 high 时不用加上 mid
        else                          //若 key>mid 则在 mid 右边的元素进行递归查找
            low=mid+1; }                //注意此时在设定 low 时不用加上 mid
    return -1 }    /*当 low>high 时表示查找区间为空，查找失败*/
```

DATE /

↑ Low ↑ Mid ↑ High

```

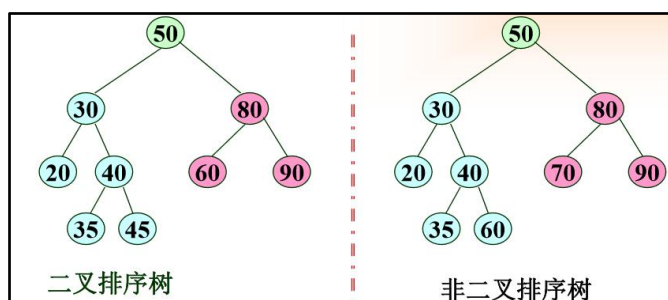
Low = 0; High = n-1
Mid = (Low + High) / 2
if (Dic.element[Mid] == Key) // 如果 Mid 的值 == key
{ Find = true;
  Pos = Mid;
}
elseif (Dic.element[Mid] < Key) // Mid 已经判断过了: 要+1
  Low = Mid + 1;
elseif (Dic.element[Mid] > Key)
  High = Mid - 1;

```

2. 动态查找——二叉排序树——二叉搜索树

(1) 特征

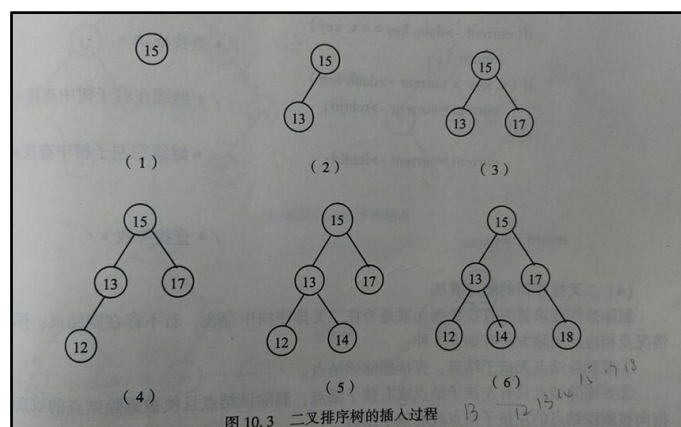
- 若左子树非空，则左子树的所有元素均小于根元素
- 若右子树非空，则右子树的所有元素均大于根元素
- 左右子树本身又各是一颗二叉排序树



(2) 二叉排序树可以通过树的中序周游获得关键字的有序序列

- 二叉排序树的生成：每读入一个元素，建立一个新的结点，若二叉排序树为空，则新插入的结点成为二叉排序树的根；若二叉排序树为非空，则将新结点的值与根结点的值相比较，如果小于根结点的值，则插入到左子树中，若大于根结点的值，则插入到右子树中；若与根结点值相等，则停止插入。

(输入序列 15 13 17 12 14 18)



(4) 二叉排序树的查找

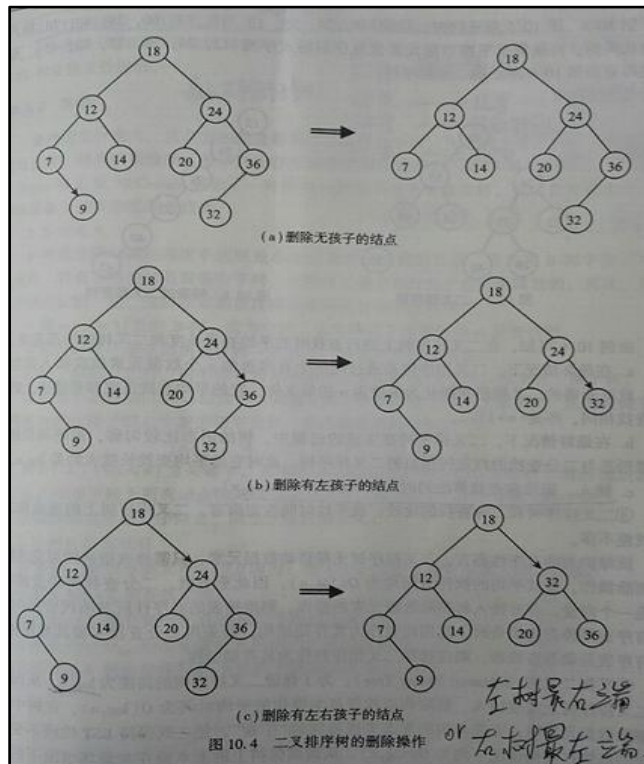
```
while (T 非空)
{ if (T.key==key)
    查到;
  else if (T.key>key)
    查左子树;
  else 查右子树;
}
```

(5) 二叉排序树的插入:

```
算法描述: if (二叉排序树中查不到该结点)
{ 建立新结点;
  if (原二叉排序树是空树)
    新结点作为根结点;
  else if (新结点<父结点)
    插入左子树;
  else 插入右子树;
}
```

(6) 二叉排序树的删除:

- 1、要删除无孩子结点，直接删除该结点
- 2、要删除只有左孩子或右孩子结点，删除该结点，且使被删除结点的双亲结点指向被删除结点的左孩子结点或右孩子结点
- 3、要删除有左右孩子的结点，首先寻找数据元素的关键字值大于要删除结点数据元素关键字的最小值，即寻找要删除结点右子树的最左结点；然后把右子树的最左结点的数据元素值拷贝到要删除的结点上；最后删除右子树的最左结点。



(7) 平均查找长度: n_i 是每层结点个数; C_i 是结点所在层次数; m 为树深。

$$ASL = \frac{1}{n} \sum_{i=1}^m n_i \cdot C_i$$

最坏情况: 插入的 n 个元素从一开始就有序 (单支树)

$$ASL = (n+1)/2$$

最好情况: 与折半查找中的判定树相同 (即形态比较均衡) 树的深度为 $\lfloor \log_2 n \rfloor + 1$

$$ASL = \log_2(n+1) - 1$$

(8) 二叉树排序树的算法性能比较:

- 二叉排序树的平均查找长度与二叉树的形态有关。N 个节点不同序可能有 $n!$ 中不同二叉树, 其中有的二叉树形态相同。
- 最坏情况下退化为深度为 n 的单支树, $ASL = (n+1)/2$
- 最好情况下, 二叉排序树的形态比较均匀, 类似于完全二叉树, $ASL = \log_2 n$, 所以需要使用平衡二叉树 (见下)。
- 二叉排序树的插入, 删除和查找算法的时间复杂度为 $O(\log_2 n)$

3. 散列法中负载因子的概念, 设计散列函数需要考虑的因素, 拉链法

3.1 散列表的定义

散列表的目的: 使用关键字直接找到记录存储地址

散列表 (哈希表): 以关键字 k 为自变量, 通过一定的函数关系 $h(k)$ 计算出对应数值的函数值, 把这个值作为 k 的存储地址, 检索时用同样的方法得到存储地址, 然后得到相应的结点。

3.2 冲突 (碰撞) 及负载因子

冲突: 是指两个不相等的关键字 k_1, k_2 , 用散列函数 $h(k)$ 得到的结果 $h(k_1) = h(k_2)$, 这种现象称为冲突。

发生碰撞的两个或多个关键码称为同义词。

负载因子:

$$\text{负载因子}(\alpha) = \frac{\text{散列表中结点数目}}{\text{基本区域能容纳的结点数}}$$

3.3 设计散列函数需要考虑的因素:

- 计算哈希表函数所需时间
- 关键字长度
- 哈希表长度 (哈希地址范围)
- 关键字分布情况
- 记录查找频率

3.4 散列表解决冲突的方法

3.4.1 开放地址法: 指为发生冲突的关键码找到哈希表中的最近的另一个尚未被使用的位置。

例: ① 开放地址法: 关键字序列为 19, 56, 23, 14, 68, 82, 70, 36, 91.
哈希表长 11, 哈希函数为 $h(k) = k \% 11$

0	1	2	3	4	5	6	7	8	9	10
	56	23	14	68	82	70	36	19	91	

① $19 \% 11 = 8$ ∴ 把 19 放在 8 中
② $56 \% 11 = 1$ ∴ 把 56 放在 1 中
③ $23 \% 11 = 1$ ∴ 把 23 放在 1 中 但 1 中有 56 ∴ 把 23 放在下一个空位 2 中
④ $14 \% 11 = 3$ ∴ 3 中
⑤ $68 \% 11 = 2$ ∴ 把 68 放在 2 中 但 2 中有 23 ∴ 把 68 放在 4 中
⑥ $82 \% 11 = 5$ 把 82 放在 5 中
以此类推 得到上图的存储方式. 23, 68, 70, 36, 91 为有冲突的元素

$ASL = (1 \times 4 + 2 + 3 + 3 + 5 + 7) / 9$

查 23 时 找 1, 再找 2, $C_1 = 2$ 查 36 时 $36 \% 11 = 3$ ∴ 先找 3, 4, 5, 6, 7 找 5 次 $C_1 = 5$

(1) 计算第 n 位置**查找成功**的平均查找长度为:

$ASL_1 = (N_1 + N_1 + \dots + N_{10}) / \text{空间大小}$, N 为每个元素查找成功的比较次数

(2) 计算第 n 位置**查找不成功**时的比较次数为, 第 n 位置到第 1 个没有数据位置的距离。找不成功的平均查找长度 $ASL_2 = (n_1 + n_2 + \dots + n_i) / A$, A 为分配的所有空间, 不是元素使用的空间 (例如 $h(k)$ 是 $\%15$, 但是元素只使用了 12 格, 则 ASL_1 除以 12, ASL_2 除以 15)

例题:

7. 已知关键字序列{19, 05, 21, 24, 45, 20, 68, 27, 70, 11, 10}, 哈希表表长为 15, 哈希函数 $H(key)=key \text{ MOD } 13$, 处理冲突的方法为线性探测法, 请给出哈希表, 并计算在等概率的条件下的查找成功和不成功时平均查找长度

7. 解答:

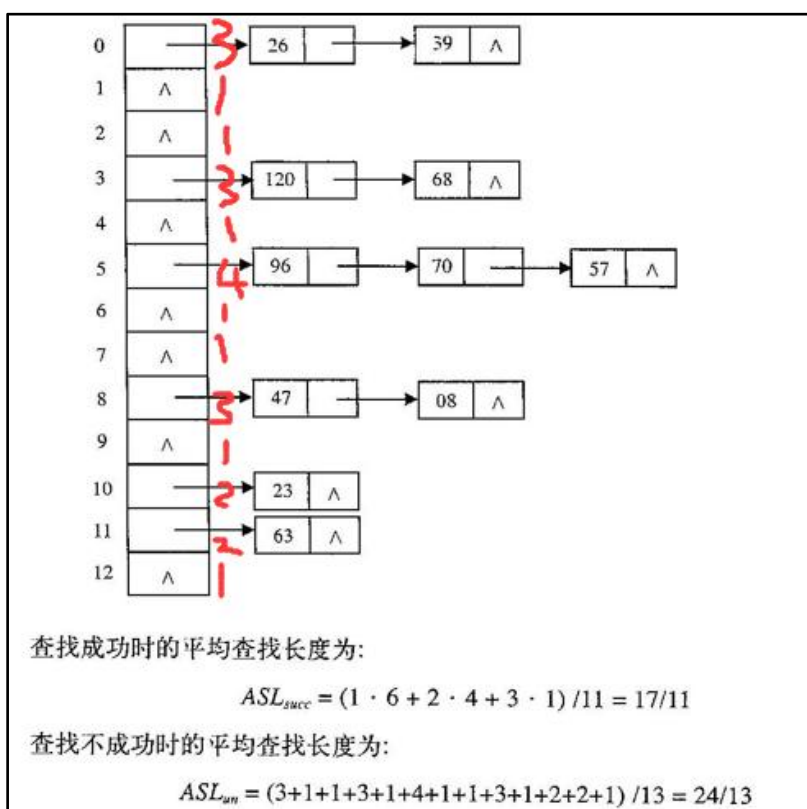
地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
元素		27		68		05	19	45	21	20	70	24	11	10	
		1		1		1	1	2	1	3	6	1	2	4	

查找成功时的平均查找长度为:

$$ASL_{succ} = (1+1+1+1+2+1+3+6+1+2+4) / 11 = 23/11$$

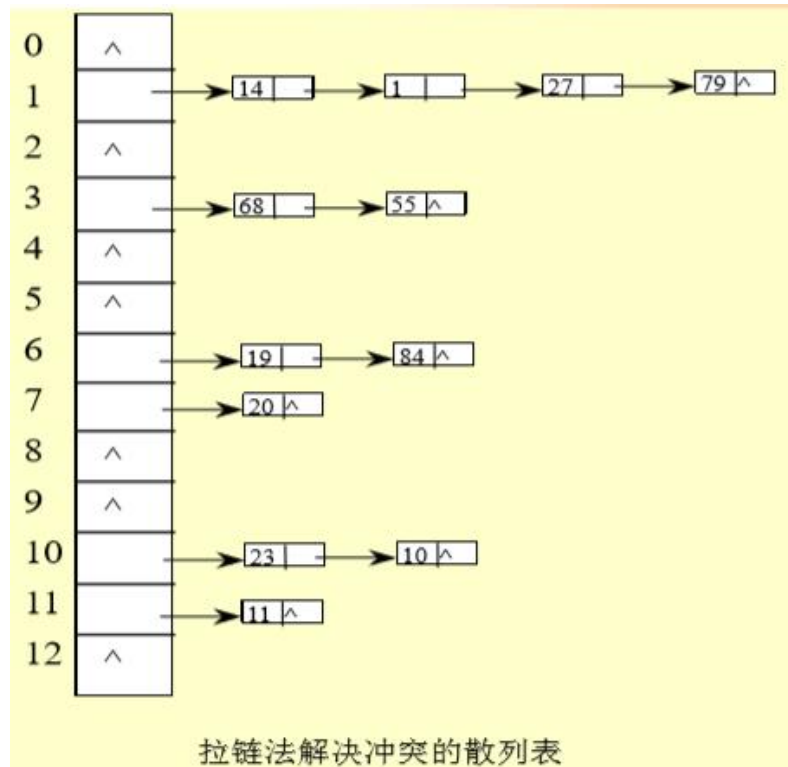
查找不成功时的平均查找长度为:

$$ASL_{un} = (1+2+1+2+1+10+9+8+7+6+5+4+3+2+1) / 15 = 62/15$$



3.4.2 拉链法: 是把相互冲突的同义词用同一个单链表链接起来, 若干组同义词可以组成若干个单链表

例题: 给定关键字序列: 19,14,23,1,68,20,84,27,55,11,10,79, $h(k)=k\%13$, 用拉链法解决冲突建立哈希表。

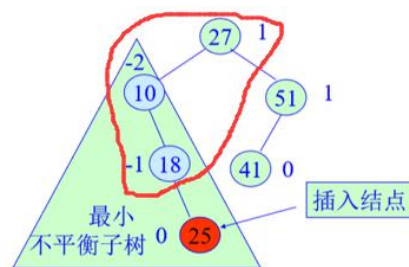


4. 二叉排序树的生成与删除，ASL 的计算

5. AVL 树的生成过程 (如何调整)，ASL 的计算

5.1 AVL 树即平衡二叉树

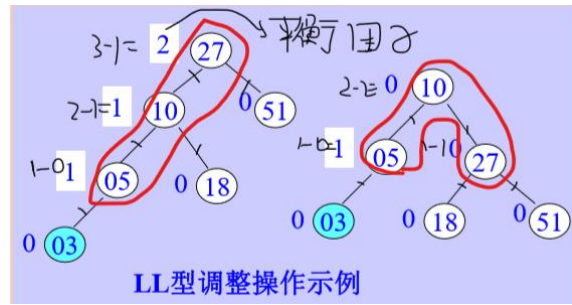
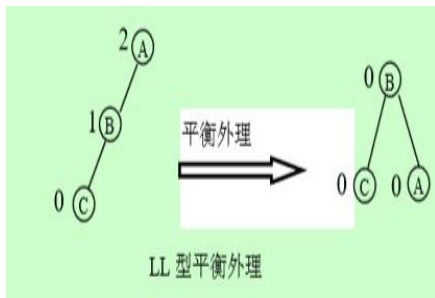
- (1) 特点：要么是一颗空树，若非空则一定满足左子树和右子树都是平衡二叉树且左右子树深度之差不大于 1。
- (2) 平衡因子：当前节点的平衡因子定义为当前节点左子树的深度减右子树的深度。
- (3) 最小不平衡子树：离插入节点最近，且平衡因子的绝对值大于 1 的节点为根的子树



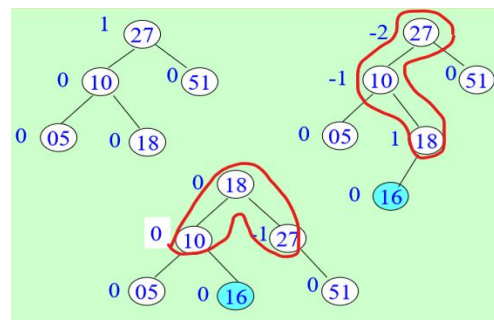
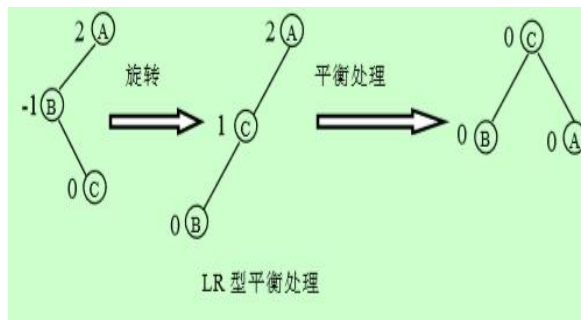
5.2 AVL 树的平衡调整

1. 目的：保证生成的二叉排序树的高度为 $\log_2 n$ ，从而使二叉排序树的插入，删除，查找等操作的平均时间为 $O(\log_2 n)$ 。所需调整的是最小不平衡子树，保证二叉树始终处于平衡状态。
2. 平衡调整模式：LL 型调整，LR 型调整，RR 型调整，RL 型调整

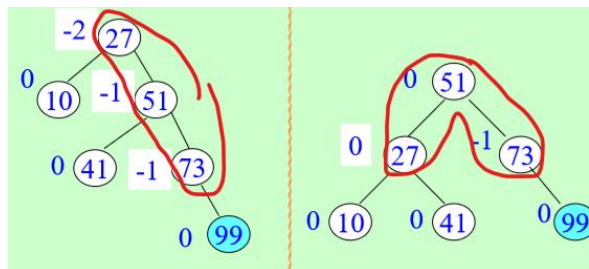
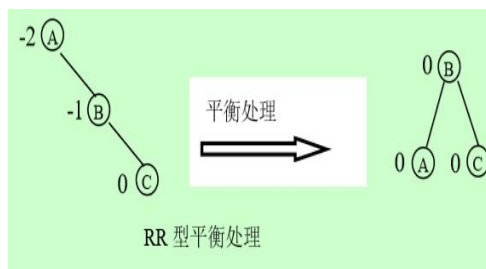
LL 型调整



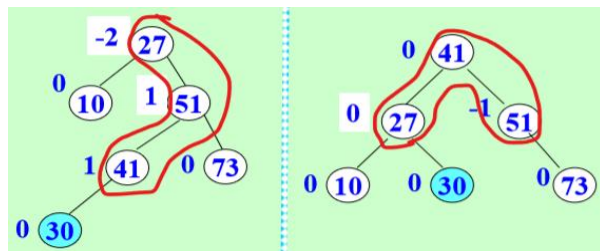
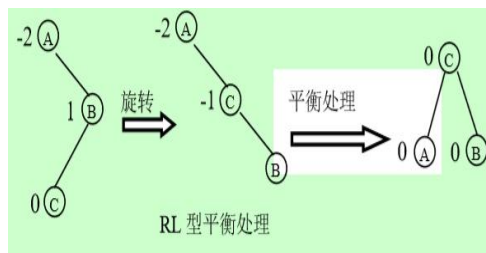
LR 型调整



RR 型调整



RL 型调整



例题：关键字序列为：10，13，19，7，4，8，15，24，33，21 试用二叉排序树和平衡二叉树进行检索，给出检索 15 时需要的比较次数及成功检索的平均检索长度。

①

= 叉排序树

$$ASL = \frac{1}{10} (1 + 2 \times 2 + 3 \times 3 + 4 \times 2 + 5 \times 2)$$

$$= 3.2$$

② AVL 树

$$ASL = \frac{1}{10} (1 + 2 \times 2 + 3 \times 4 + 4 \times 3)$$

$$= \frac{1}{10} \times 29$$

$$= 2.9$$

③ 最佳二叉排序树不作要求

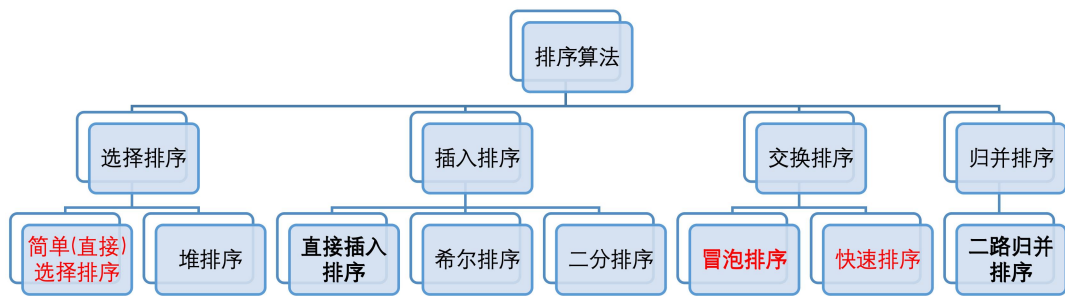
第 8 部分

1. 排序的方法及比较。堆的定义

1.1 排序算法优劣的判断标准

时间复杂度 (最重要)	空间复杂度 (重要)	稳定性
最坏是 $O(n^2)$ 单支树	排序过程中使用的辅助空间	算法按关键字多次排序得到结果相同则稳定，否则不稳定
最好是 $O(\log_2 n)$ 看做完全一棵二叉树	最好情况 $O(1)$, 最坏情况 $O(n)$	常用的只有直接插入排序，冒泡排序和归并排序是稳定的

1.2 排序算法的分类



1.3 各种算法的思想及实例

1) 直接选择排序算法

思想：第一趟：从 n 个元素中找出关键字最小的记录放在第一个位置

第二趟：从元素集合的 $2 \sim n$ 个元素中找出最小关键字记录放在第二个位置

第三趟：..... $3 \sim n$三.....

以此类推直到全部遍历

② 算法实例

	46	79	56	38	40	84
第1趟：	38	79	56	46	40	84
第2趟：	38	40	56	46	79	84
3	38	40	46	56	79	84
4	38	40	46	56	79	84
5	38	40	46	56	79	84
6	38	40	46	56	79	84

要求写出代码：

```
int i,j,pos;int temp;
int a[]; //存放 n 个无序元素的数组
for(i=0;i<n-1;i++){
    pos=i;
    for(j=i+1;j<n;j++){
        if(a[j].key<a[i].key)
            pos=j; //找出序列中的最小值
    }
    if(pos!=i){
        temp=a[i]; //交换该最小值和 i 的位置
        a[i]=a[pos];
        a[pos]=temp;
    }
}
```

算法评价：直接选择排序共需 $n-1$ 次选择，每次选择要进行 $n-i$ 次比较，所以算法的比较次数为 $C=(n^2-n)/2$ ，时间复杂度为 $O(n^2)$ ，算法不稳定

原始数据越有序，需要移动元素的次数越小，选择单链表的存储方式，因为要经常交换元素。

D。

A、直接插入排序

B、起泡排序

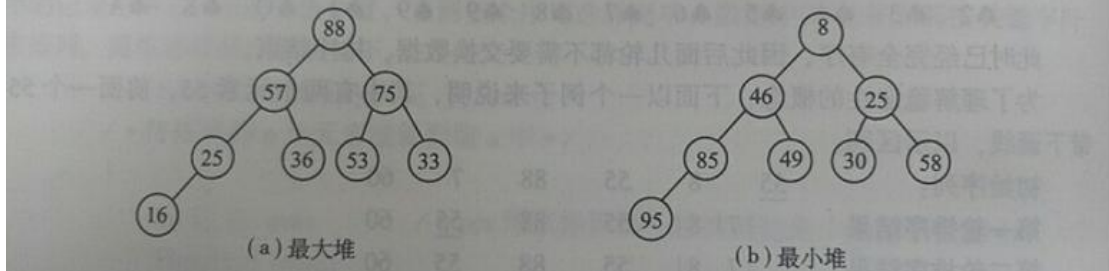
C、快速排序

D、直接选择排序

2) 堆排序算法

堆的定义:

若以一维数组 a 存储一个堆, 把这个数组看成一棵完全二叉树, 则堆对应一棵完全二叉树, $a[0]$ 是树根, $a[1]$ 是根结点的左孩子, $a[2]$ 是根结点的右孩子, 依此类推。这棵二叉树中所有非叶结点的值均不大于 (或不小于) 其孩子的值, 根结点的值是最小 (或最大) 的。



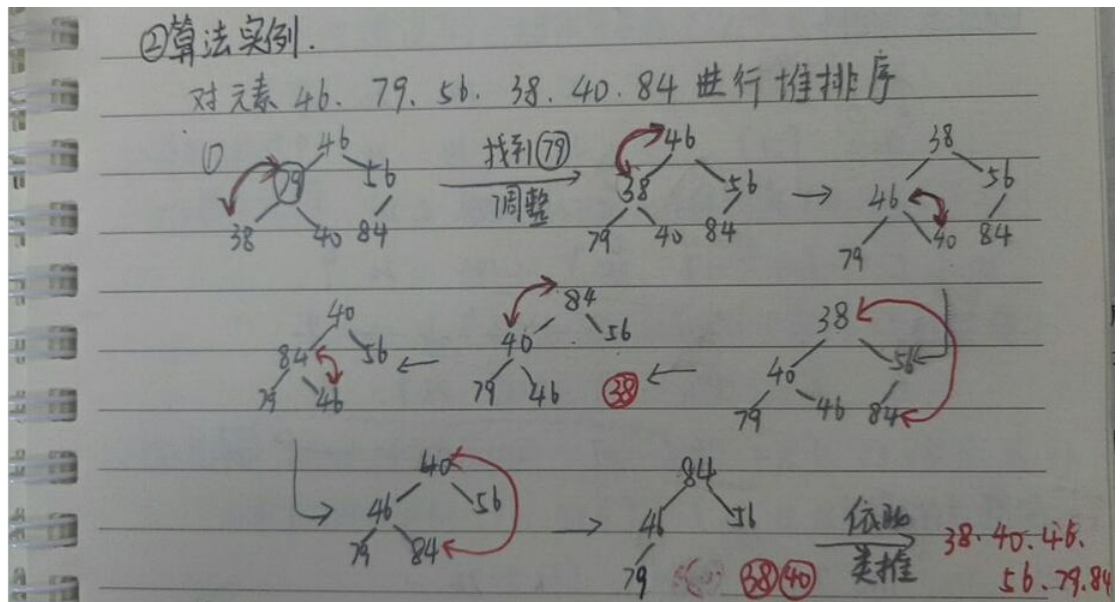
思想：第一步：把无序的 n 个元素建成一个完全二叉树

第二步：调整完全二叉树为小根堆或大根堆

第三步：输出堆顶元素，调整剩下的元素

第四步：循环直至所有元素输出

实例:



算法评价: 1、在堆排序中共需进行 $n[n/2]-1$ 次筛选

2、每次输出堆顶元素进行调整时的时间复杂度平均是 $O(\log_2 n)$, 对 n

各节点进行堆排序的时间复杂度为 $O(n \log_2 n)$, 空间复杂度为 $O(1)$

3.要求从小到大排序则建立最大堆, 要从大到小排序则建立最小堆

4.可以用顺序存储创建堆

3) 直接插入排序算法

思想：第一步：对 n 个元素假定以第一个元素是有序序列，比较 $a[0], a[1]$ 进行排序，形成 $\{a[0], a[1]\}$ 的一个有序序列

第二步：再将 $a[2]$ 与 $\{a[0], a[1]\}$ 进行排序，形成有序序列 $\{a[0], a[1], a[2]\}$
以此类推，直至找出包含所有元素的有序序列 $\{.....\}$

②算法实例

初:	17	3	25	14	20	9
a:	{17}	3	25	14	20	9
b:	{3, 17}	25	14	20	9	
c:	{3, 17, 25}	14	20	9		
d:	{3, 14, 17, 25}	20	9			
e:	{3, 14, 17, 20, 25}	9				
f:	{3, 9, 17, 20, 25}					结束

b 算作第一趟，c 算作第二趟，共需进行 $n-1$ 趟排序

- 算法评价：
- 1、最好情况下：总比较次数： $n-1$ ，总移动次数 $2(n-1)$ ，时间复杂度 $O(n)$
 - 2、最坏情况，总比较次数： $(n-1)(n-2)/2$ ，总移动次数 $(n-1)(n-4)/2$ ，时间复杂度 $O(n^2)$
 - 3、平均情况下，时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(1)$ ，算法稳定

4.原始数据越有序，比较次数越少，更适合有序数据，是稳定排序算法

4) 希尔排序算法

思想：第一步：先把 n 个元素分成若干组，选择步长序列 $t_1, t_2, \dots, t_k, t_1=n/2, \dots, t_k=1$

第二步：将原序列分成若干个序列，分别对各子序列进行直接插入排序

实例：

②实例 取 $d=4$ ，原序列为 16, 25, 12, 30, 47, 11, 23, 36, 9, 18, 31

1) 取步长为4的序列 {16 47 9} {25 11 18} {12 23 31} {30 36}

2) 再取每个序列的最小值 9 11 12 30 16 18 23 30 47 25 31

3) 第2趟 $d=2$ 的序列 {9 12 16 23 47 31} {11 30 18 36 25}

4) 再进行排序 9 11 12 30 16 18 23 36 47 25 31

算法评价：时间复杂度 $O(n(\log_2 n)^2)$ ，空间复杂度 $O(1)$ ，不稳定

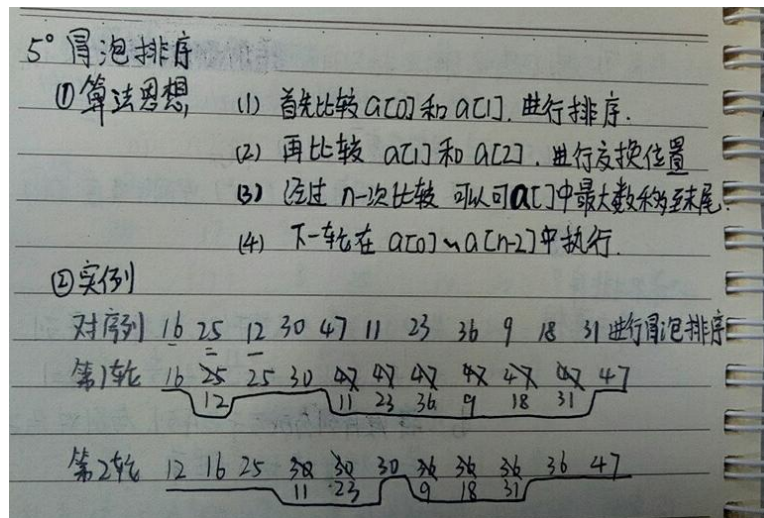
希尔排序是强力版的直接插入排序，元素数 n 越大，算法的优势越明显，需要知道元素的位置，不能使用链式存储结构，适用于顺序存储

5) 冒泡排序算法

两两比较，调整顺序， $a[0]$ 与 $a[1]$ 比再调整， $a[1]$ 与 $a[2]$ 比再调整， $a[2]$ 与 $a[3]$ 比再调整，以此类推，比到结尾就是一轮。

算法适用于无序的数据，比较次数与原始数据的有序性无关，可以顺序存储也

可以链式存储。



- 算法评价：
- 1、最好的情况是原始数据有序，循环 $n-1$ 次，不发生交换，时间复杂度为 $O(n)$ ，最坏的时间复杂度为 $O(n^2)$
 - 2、空间复杂度为 $O(1)$
 - 3、冒泡排序是稳定的排序算法之一。三个常见排序算法中稳定的算法是：冒泡排序，直接插入排序，归并排序。
 - 4、最好的情况是原序列有序，则只需要扫描一次，时间复杂度是 $O(n)$ ，最坏的情况是要比较 $n-1$ 次，时间复杂度是 $O(n^2)$

例题：

4. 对 n 个记录进行冒泡排序时，最少的比较次数为 $n^2 - n - 1$ ，最少的趟数为 $n - 1$ 。

要求会写算法代码：

```
int a[], i, j;
tag = 1;
for(i = 1; i < n && tag == 1; i++) {
    tag = 0;
    for(j = 0; j < n - i; j++) { // 只需和前面的 n-i 个数进行比较末尾已经确定的数不参与
        if(a[j].key < a[j+1].key) {
            tag = 1; //
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}
```

例题：

9. 对 n 个元素的序列进行起泡排序时，最少的比较次数是 $1 + n - 1$ 。
10. 采用顺序查找方法查找长度为 n 的线性表时，每个元素的平均查找长度为 $(n+1)/2$ 。

6) 快速排序算法

适用于顺序存储，方便存储元素，不适合链式存储，因为单链表只能从开头逐渐

遍历到某个位置，不能直接查找某一个具体的元素。

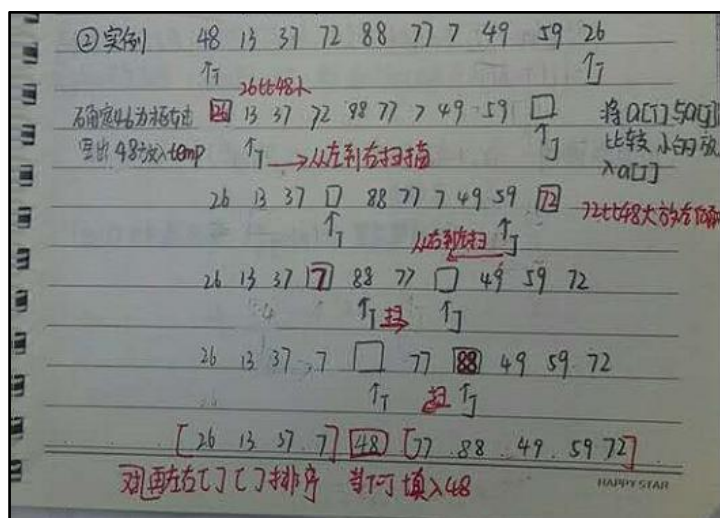
- 1) 算法思想：
 - (1) 选取一个标准元素（通常是第一个元素——枢轴）
 - (2) 在序列的右端开始扫描，遇到比 $a[0]$ 小的元素 $a[j]$ 则放到 $a[0]$ 的位置，再从左端开始扫描，遇到比 $a[0]$ 大的元素则放到 $a[j]$ 的位置，以此类推直至 $i=j$
 - (3) 当左右指针 i 和 j 指向同一个位置时放入枢轴元素
 - (4) 采用同样的方法对枢轴的左右子序列进行快速排序

- 2) 算法代码：

```
void QuickSort(int a[],int low,int high){
    int i=low,j=higt,temp;
    temp=a[low];
    while(i<j){
        while(i<j&&temp.key<=a[j].key){
            j--; } //从右到左开始扫描，若一直满足比 temp 大则无需交换不中断循环，while 循环结
                //束后，j 指向那个比 temp 大的数
        if(i<j){
            a[i]=a[j];i++; } //这个 if 执行数据交换
        while(i<j&&a[i].key<temp.key){i++; } //再从左端开始扫描，当循环结束时
                //temp 指向一个比 temp 大的数，需要交换
        if(){
            a[j]=a[i];j--; }
        } //这个 if 执行数据交换
    a[i]=temp; //快速排序循环结束后 i=j, low 和 high 不变
    if(low<i) QuickSort(a,low,i-1); //对左端子集合进行递归
    if(i<higt) QuickSort(a,j+i,high); //对右端子集合进行递归
}
```

- 3) 算法评价：快速排序不稳定。

时间复杂度为 $O(n \log_2 n)$ ，最坏的时间复杂度为 $O(n^2)$



7) 二路归并排序算法

7°二路归并排序

①思想

- (1) 对元素两两归并，得长度为2的序列。
- (2) 再两两归并得长度为4的序列。

②实例

初始序列: 46 55 13 42 94 105 17 70

一趟: [46 55] [13 42] [94 105] [17 70]

二趟: [13 42 46 55] [17 70 94 105]

三趟: [13 17 42 46 55 70 94 105]

③效率

- a. 时间复杂度 $O(n \log_2 n)$
- b. 空间 $O(n)$
- c. 稳定，常用于外排序。

1.4 各种算法的性能比较:

排序方法	最好时间	平均时间	最坏时间	最坏空间	稳定性
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	不稳定
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序		$O(n(\log_2 n)^2)$		$O(1)$	不稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n)$	不稳定
归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	稳定
基数排序 (链式队列)	$O(mn)$	$O(mn)$	$O(mn)$	$O(n)$	稳定

2. 给出排序序列，要求会用堆排序、SHELL 排序，快速排序或归并排序写出第一、二趟的结果

3. 会写冒泡排序、直接选择排序，快速排序算法